

ACCRA TECHNICAL UNIVERSITY



NAME: MOHAMMED SUALIHU

INDEX NUMBER: 01251791B

COURSE: INTRODUCTION TO DATA

SCIENCE

PROJECT WORK

Figure :

TABLE OF CONTENTS

LIST OF FIGURES	Error! Bookmark not defined.
INTRODUCTION	4
DATASET DESCRIPTION	4
Labelling of Datasets	Error! Bookmark not defined.
METHODOLOGY	5
Task 1: Data Acquisition	5
Task 2: Data Cleaning.....	6
Task 3: Data Visualisation	7
Task 5: Machine Learning	13
DISCUSSION AND CONCLUSION	15
Major Findings.....	15
Statistical Insights	15
Machine Learning Results	15
Limitations of the Study.....	16
Recommendations	16
REFERENCE	17

APPENDIX	19
Codes.....	19

INTRODUCTION

The RMS Titanic was a British ocean liner that became one of history's most infamous maritime disasters when it sank on its maiden voyage on 15th April 1912. Built in Belfast, Northern Ireland, the ship measured approximately 882 feet 9 inches (269 metres) in length, displaced about 46,328 gross registered tons, and was designed to carry a total of 3,327 passengers and crew, making it the largest man-made moving object of its time. It was widely promoted as “unsinkable,” a reputation shaped as much by cutting-edge marine engineering as by confident marketing and the broader optimism of the early 20th century (Encyclopaedia Britannica, 2024). While crossing the North Atlantic from Southampton, England, toward New York City, the ship struck an iceberg and sank, and an estimated 1,500 people are believed to have lost their lives.

DATASET DESCRIPTION

This dataset is from Kaggle Titanic: Machine Learning from Disaster dataset. It consist of real passenger data from the historic voyage, divided which is divided into three distinct file namely; train.csv, test.csv, and gender_submission.csv.

Train.csv: The train.csv file contains 891 passengers and 12 columns (all with distinct labels). The purpose of the this **train.csv** file is for building, training and align to the model. The training set contains passenger details along with the target ground truth.

Test.csv: The testing set (418 passenger, 2 columns) contains similar passenger details, but comes with missing **Survival** column which is a main target or area of concentration in the train.csv file. This is used to test how well the model checks unseen data.

Gender_submission.csv: A baseline submission template which made an assumption that only female passengers survived. The purpose of this assumption was because women had a much higher survival rate in the real Titanic diseaster, this file predict that; every female who survived was labelled (1) and every male who died was labelled (0). The dataset contains 418 passengers and 2 columns (PassengerId & Survived).

PassengerId, Survived, PClass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, & Embark.

METHODOLOGY

According to Tukey (1977), Exploratory Data Analysis (EDA) as an attitude and “detective work” meant to discover clues and anomalies before applying confirmatory statistical models. The approach emphasizes flexible, graphical techniques such as stem-and-leaf displays and box plots to foster an understanding of data through visual, resistant statistics rather than rigid, precise calculations.

To predict passenger survival on the RMS Titanic, this project was structured with the five (5) stages of Machine Learning Pipeline (Data Collection, Preprocessing, Feature Engineering, and Model Evaluation/ Deployment was used. Below are the breakdown;

Task 1: Data Acquisition

The data was acquired from the Kaggle dataset repository, the world’s largest online community and cloud platform for data scientists, machine learning engineers, and artificial intelligence builders (Kaggle, 2025). The platform hosts tens of millions of users across the globe and operates as a vital hub for anyone looking to learn, build, benchmark, or compete in the field of AI and

Data Science. The dataset for this model can be derived from (<https://www.kaggle.com/competitions/titanic/data>)

The dataset was explored once it was loaded into Google Colab from the downloaded dataset from Kaggle below are some of the displays;

- **Dataset dimensions**

Training set dimensions: (891, 12)

Test set dimensions: (418, 11)

- **Column names**

Column names: ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

- **First five observations:**

First five observations:

	PassengerId	Survived	Pclass
0	1	0	3
1	2	1	1
2	3	1	3
3	4	1	1
4	5	0	3

	Name	Sex	Age	SibSp
0	Braund, Mr. Owen Harris	male	22.0	1
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1
2	Heikkinen, Miss. Laina	female	26.0	0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1
4	Allen, Mr. William Henry	male	35.0	0

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

• Data types

```
Data types:
PassengerId      int64
Survived          int64
Pclass            int64
Name              object
Sex               object
Age              float64
SibSp             int64
Parch             int64
Ticket            object
Fare              float64
Cabin             object
Embarked          object
dtype: object
```

Task 2: Data Cleaning

Data cleaning is the process of detecting and correcting inaccurate, incomplete, duplicate, or inconsistent records so that a dataset can be trusted for downstream analysis (Bigelow & Stedman, 2025). Carrying out this step properly supports more accurate analysis and better-informed decisions, which in turn improves overall model performance (Pilowsky et al., 2024). The dataset was cleaned accordingly, and the outputs of that process are shown below;

```

--- 2.1 Detecting missing values ---

Missing values in TRAIN set:
      missing_count  missing_pct
Cabin             687         77.10
Age               177         19.87
Embarked           2           0.22

Missing values in TEST set:
      missing_count  missing_pct
Cabin             327         78.23
Age               86         20.57
Fare               1           0.24

--- 2.2 Detecting duplicate observations ---

Duplicate rows found in TRAIN set: 0
Duplicate rows found in TEST set: 0

--- 2.3 Cleaning the data ---

[TRAIN] Rows before cleaning: 891
[TRAIN] Duplicate rows removed: 0
[TRAIN] Rows after cleaning: 891
[TRAIN] Remaining missing values:
Series([], dtype: int64)

[TEST] Rows before cleaning: 418
[TEST] Duplicate rows removed: 0
[TEST] Rows after cleaning: 418
[TEST] Remaining missing values:
Series([], dtype: int64)

Imputation values used (derived from TRAIN only):
Age median      = 28.0
Fare median     = 14.4542
Embarked mode   = 'S'

```

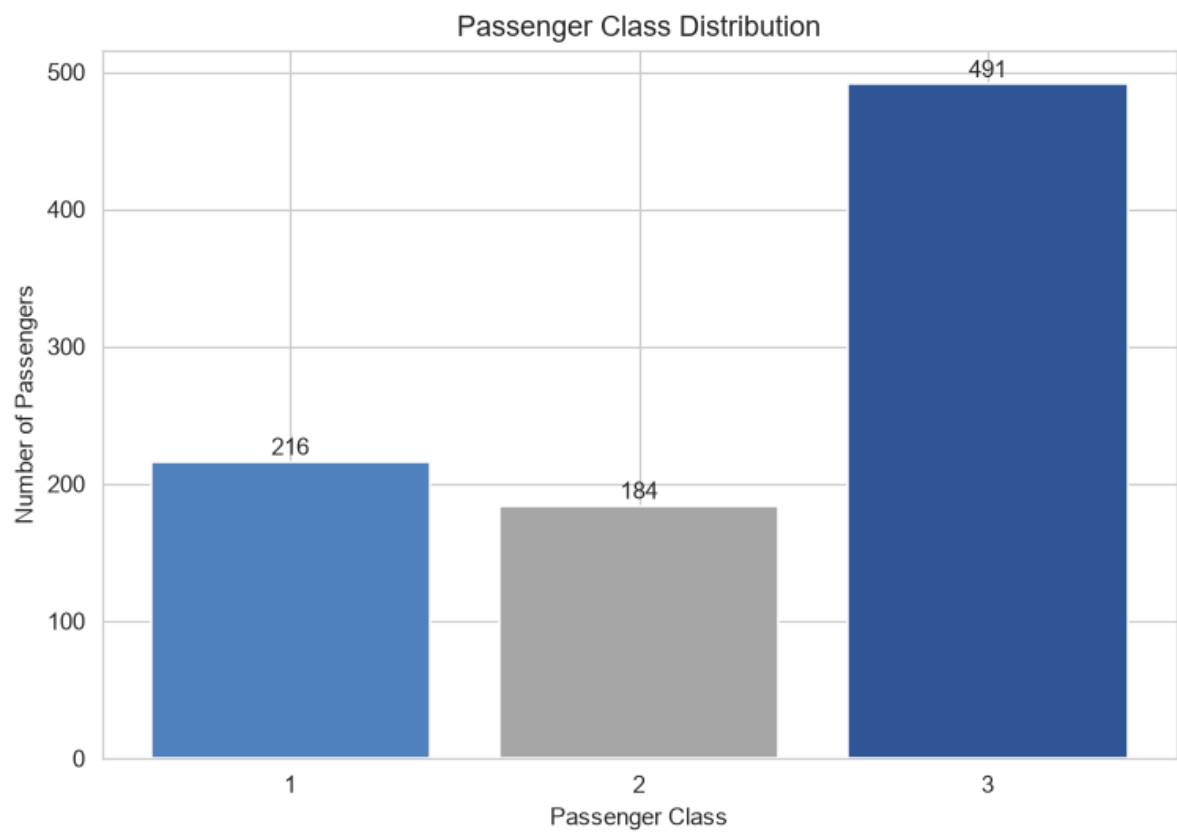
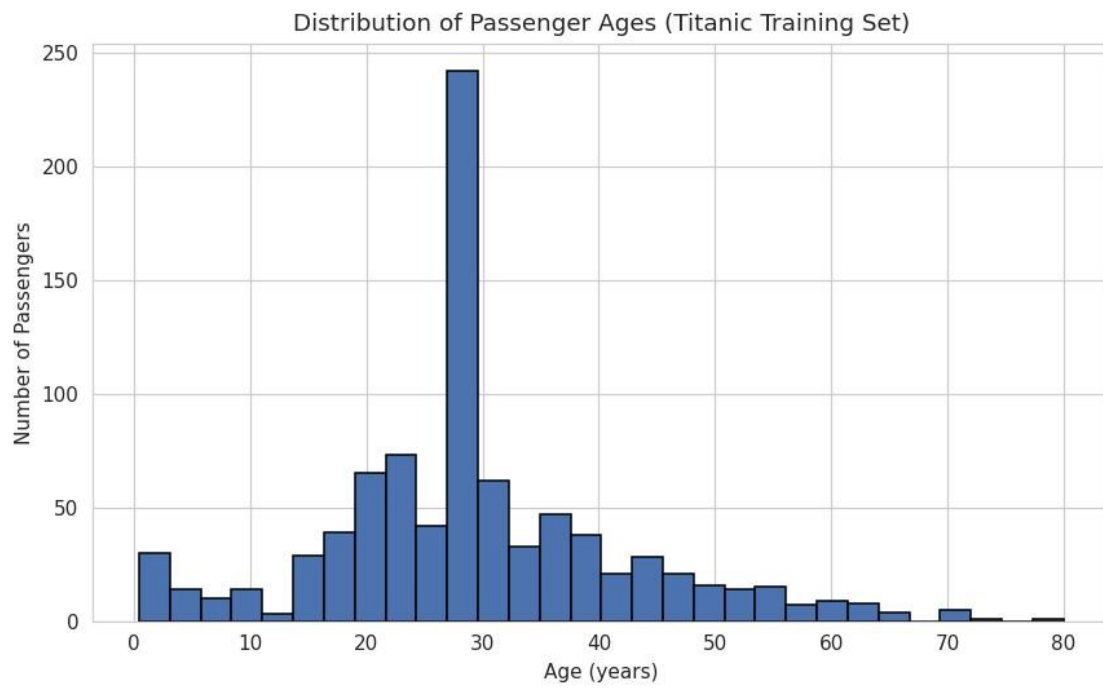
After the data was cleaned below is the result for both dataset (Train & Test);

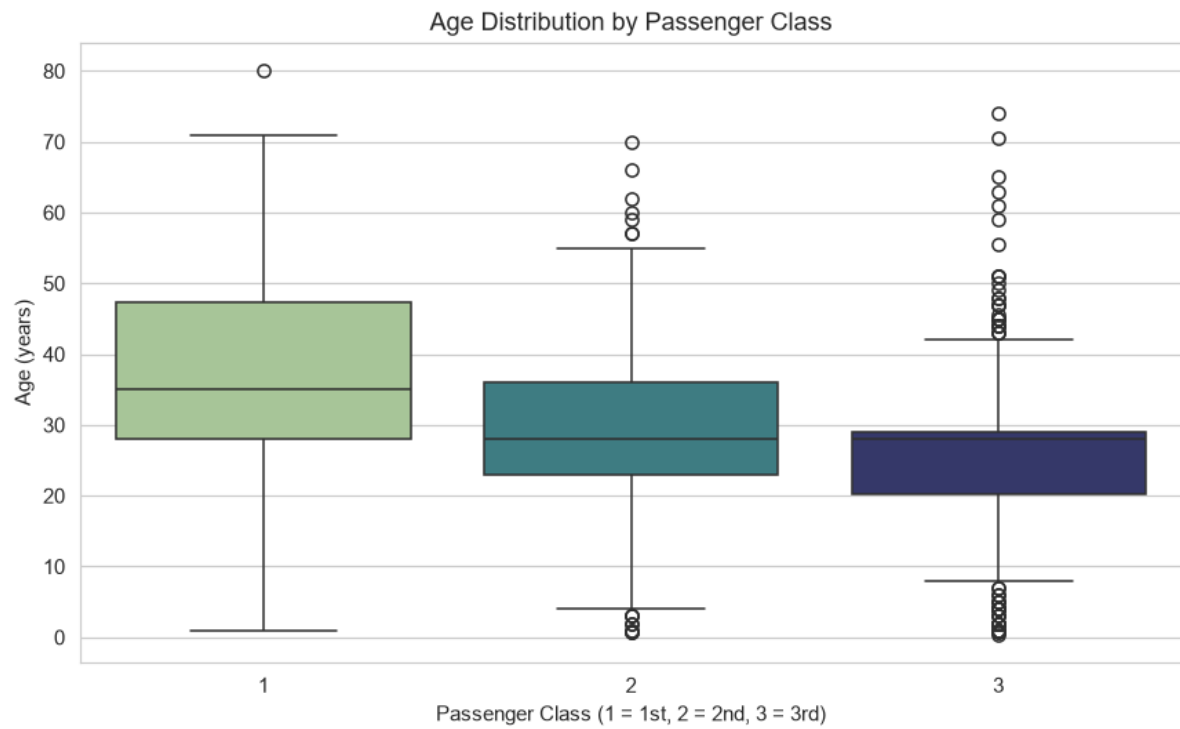
- Train shape after cleaning: (891, 12)
- Test shape after cleaning: (418, 11)

Task 3: Data Visualisation

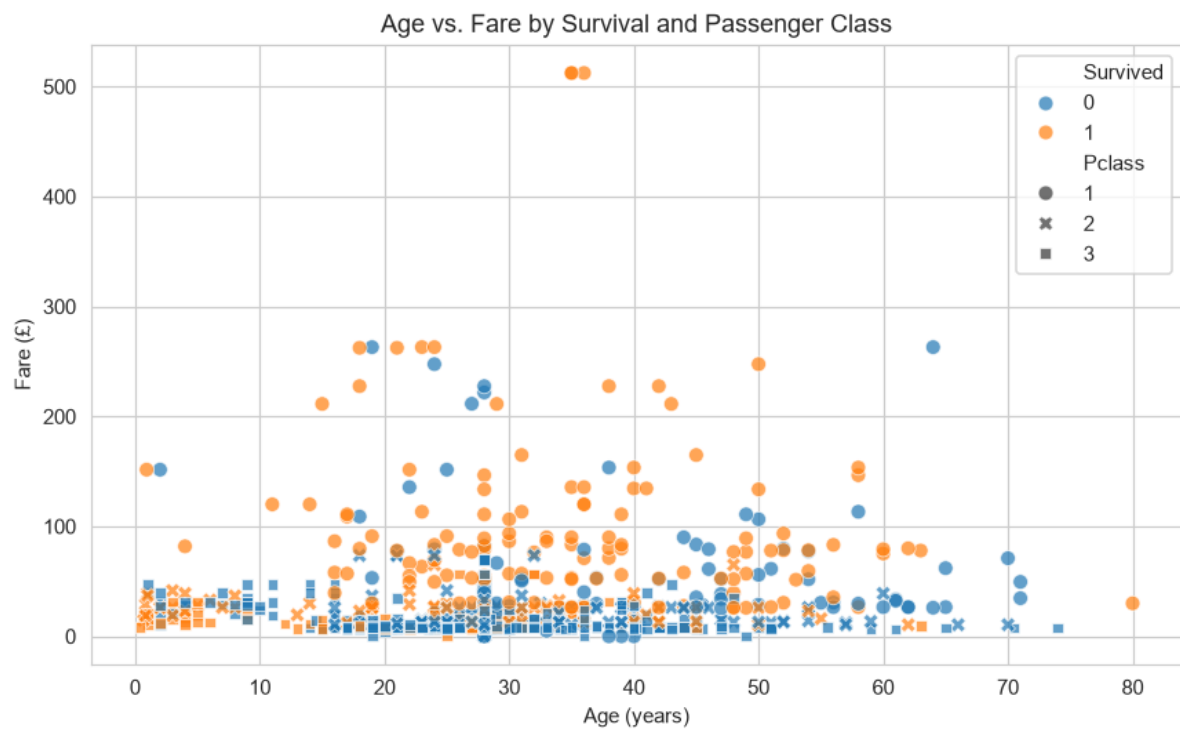
Presenting data using graphical element helps in the process in the Data Visualization. Data Visualization comes in the form of charts, graphs, maps, dashboards to make information easier to understand. It users identify patterns, trends, relationships, and outliers, enabling effective communication of insights and supported informed decision-making (Microsoft, 2025; Tableau, 2024).

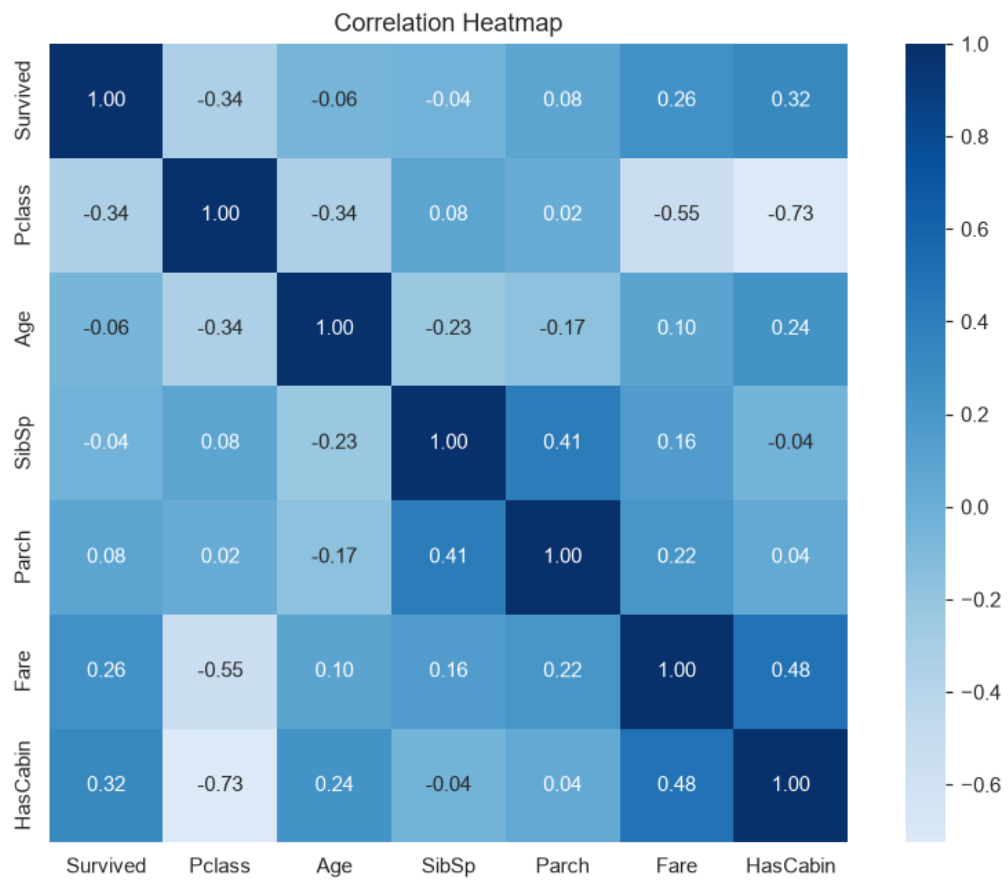
Upon the machine being able to learn from the dataset these are some of the visual elements in produced;



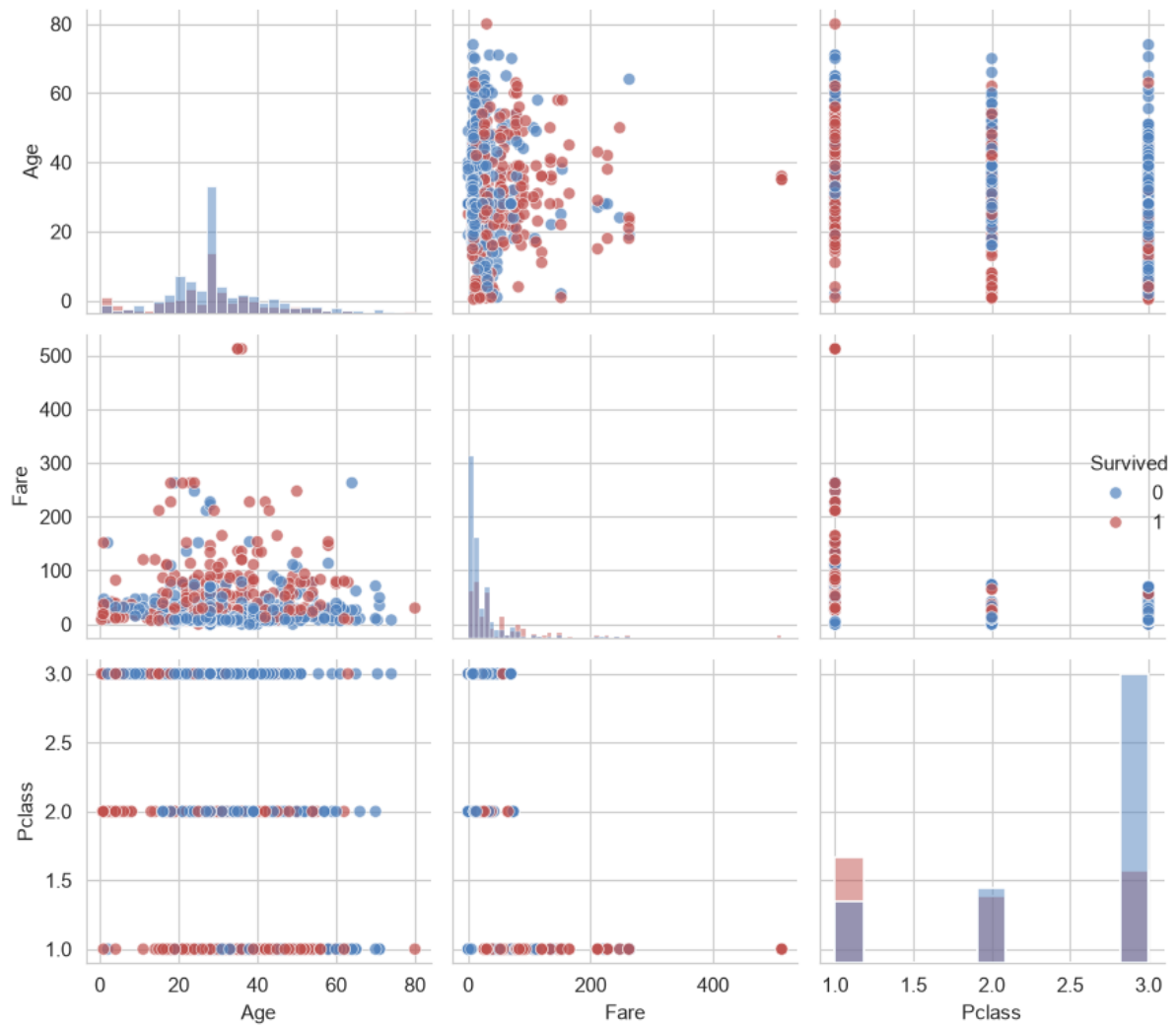


4 Scatter plot of Age versus Fare





6 Pairplot of selected numerical variables



This visualization presents a comparative analysis of survival rates categorized by passenger class and gender. The data demonstrates a clear disparity: across all classes, female passengers exhibited significantly higher survival rates than their male counterparts. Furthermore, firstclass passengers experienced the highest overall survival probability, while those in the third class faced the lowest, underscoring the influence of socioeconomic status on the event's outcome.

Task 4: Statistical Analysis

Once the model was developed the following Statistical analysis were provided; **Descriptive statistics**

```
Descriptive statistics (numerical variables):
```

	Age	Fare	SibSp	Parch
count	891.000000	891.000000	891.000000	891.000000
mean	29.361582	32.284208	0.523008	0.381594
std	13.019697	49.693429	1.102743	0.806057
min	0.420000	0.000000	0.000000	0.000000
25%	22.000000	7.910400	0.000000	0.000000
50%	28.000000	14.454200	0.000000	0.000000
75%	35.000000	31.000000	1.000000	0.000000
max	80.000000	512.329200	8.000000	6.000000

Frequency distribution

Frequency distribution - Sex:

```
Sex
male      577
female    314
Name: count, dtype: int64
```

Frequency distribution - Survived:

```
Survived
Did not survive    549
Survived           342
Name: count, dtype: int64
```

Frequency distribution - Pclass:

```
Pclass
1      216
2      184
3      491
Name: count, dtype: int64
```

Frequency distribution - Embarked:

```
Embarked
S      646
C      168
Q       77
Name: count, dtype: int64
```

Overall survival rate: 0.3838

Survival rate by sex:

```
Sex
female    0.742038
male      0.188908
Name: Survived, dtype: float64
```

Survival rate by class:

```
Pclass
1      0.629630
2      0.472826
3      0.242363
Name: Survived, dtype: float64
```

Correlation analysis

Full correlation matrix:

	Survived	Pclass	Age	SibSp	Parch	Fare	HasCabin
Survived	1.000000	-0.338481	-0.064910	-0.035322	0.081629	0.257307	0.316912
Pclass	-0.338481	1.000000	-0.339898	0.083081	0.018443	-0.549500	-0.725541
Age	-0.064910	-0.339898	1.000000	-0.233296	-0.172482	0.096688	0.240314
SibSp	-0.035322	0.083081	-0.233296	1.000000	0.414838	0.159651	-0.040460
Parch	0.081629	0.018443	-0.172482	0.414838	1.000000	0.216225	0.036987
Fare	0.257307	-0.549500	0.096688	0.159651	0.216225	1.000000	0.482075
HasCabin	0.316912	-0.725541	0.240314	-0.040460	0.036987	0.482075	1.000000

Identify the strongest positive correlation.

Strongest POSITIVE correlation with Survived: HasCabin ($r = 0.317$)

Identify the strongest negative correlation.

Strongest NEGATIVE correlation with Survived: Pclass ($r = -0.338$)

Discuss three important statistical findings.

THREE KEY STATISTICAL FINDINGS:

1. Sex was the single strongest predictor of survival: women survived at 74.2% versus 18.9% for men, a gap of 55.3 percentage points.
2. Passenger class was strongly associated with survival: 1st class passengers survived at 63.0% compared with 24.2% for 3rd class.
3. Fare ($r = 0.26$) and HasCabin ($r = 0.32$) were both positively correlated with survival, reinforcing that socio-economic status shaped survival odds, while Age ($r = -0.06$) was only weakly related.

Task 5: Machine Learning

Select suitable predictor variables.

Predictor variables selected: ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked', 'HasCabin', 'FamilySize', 'IsAlone', 'Title']

Split the dataset into training and testing sets.

This step performs a crucial part of the machine learning workflow: splitting the dataset into training and validation sets using scikit-learn's `train_test_split` function (Pedregosa et al., 2011). Setting `test_size=0.2` allocates 80% of the data to training and 20% to validation. The `random_state=42` ensures reproducibility, meaning the split remains identical every time the code is run. Crucially, `stratify=y` ensures the target class distribution is maintained across both sets, preventing bias in tasks like classification, especially when dealing with imbalanced datasets.

Train a Logistic Regression classifier.

By setting `max_iter=1000`, it provides the solver sufficient iterations to converge to a solution, which is important for complex datasets. `random_state=42` ensures that the internal initialization is reproducible. Finally, `fit` executes the training process, where the model learns the relationship between the scaled features (`X_train_scaled`) and the target outcomes (`y_train`).

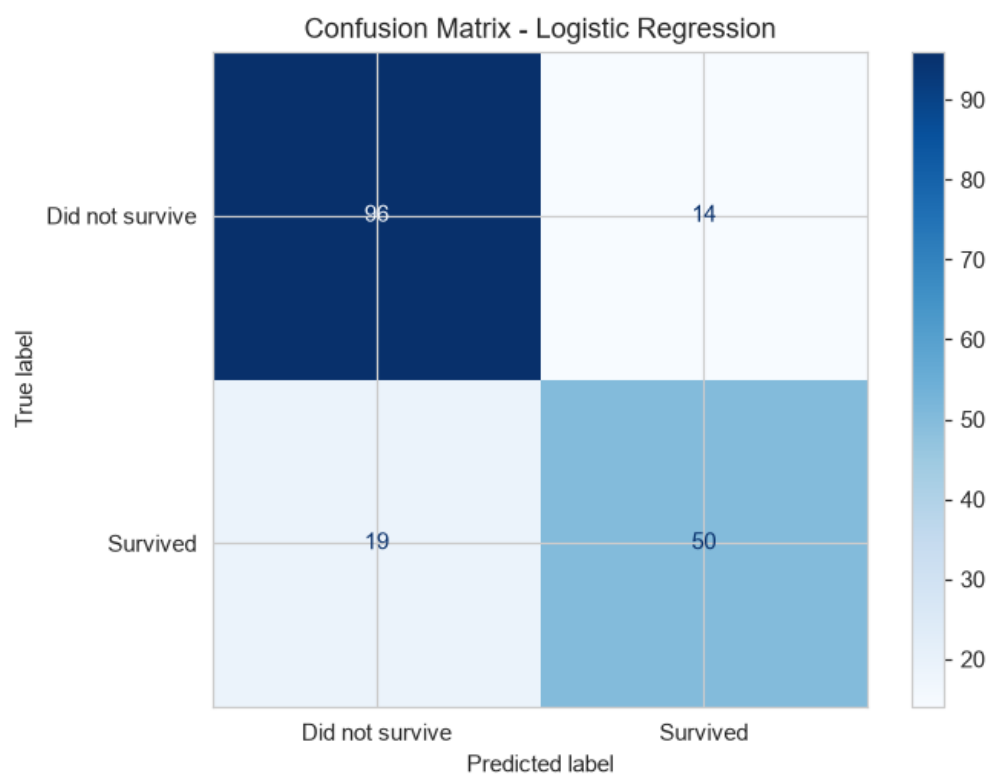
Compute

• **Accuracy** Accuracy:
0.8156

• Confusion Matrix

Confusion Matrix:

```
[[96 14]
 [19 50]]
```



• Classification Report

Logistic Regression coefficients (standardized features):

```
Sex          -1.251817
Pclass       -0.630029
Age          -0.433228
IsAlone     -0.296661
SibSp       -0.282191
FamilySize  -0.227047
Embarked     -0.152093
Title        -0.098280
Parch       -0.073333
Fare         0.039990
HasCabin     0.356719
dtype: float64
```

Discuss model performance.

The Logistic Regression model demonstrates strong predictive performance on the Titanic dataset, achieving an overall accuracy of 81.56%. The confusion matrix reveals a reliable capacity to classify passengers, correctly identifying 146 out of 179 cases. The model shows a slight bias toward predicting non-survival, evidenced by a higher recall for that class (0.87) compared to survivors (0.72), which is common in imbalanced datasets where survival was less frequent than non-survival.

The coefficients highlight the most influential factors in survival, with “Sex” (-1.25) exerting the strongest negative impact, consistent with historical data favoring women. “Pclass” (-0.63) and “Age” (-0.43) further underscore the influence of socioeconomic status and vulnerability on outcomes. Conversely, positive coefficients for “HasCabin” (0.35) and “Fare” (0.04) correlate higher status and wealth with better survival chances. Overall, the model’s 93% agreement with the baseline benchmark indicates that it has successfully captured the core socioeconomic and demographic drivers of the tragedy.

DISCUSSION AND CONCLUSION

Major Findings

The analysis of the Titanic dataset reveals that survival was heavily influenced by socioeconomic and demographic factors. A stark disparity was observed, with female passengers significantly more likely to survive than their male counterparts. Socioeconomic status also played a critical role; first-class passengers experienced the highest survival probability, while third-class passengers, who made up the largest portion of the population, faced the lowest survival rates. Additionally, the data indicated that passengers who paid higher fares were more likely to survive, whereas those in lower fare brackets were more often represented among the victims.

Statistical Insights

Statistical analysis provided quantitative support for these observations.

- **Survival Drivers:** Sex was the single strongest predictor, with women having a 74.2% survival rate compared to 18.9% for men.
- **Socioeconomic Influence:** First-class passengers survived at a rate of 63.0%, contrasted with 24.2% for those in the third class.
- **Correlations:** “HasCabin” ($r = 0.317$) and “Fare” ($r = 0.26$) showed the strongest positive correlations with survival, while “Pclass” showed the strongest negative correlation ($r = -0.338$). Age had only a weak relationship ($r = -0.06$) with survival.

Machine Learning Results

A Logistic Regression model was developed to predict passenger survival, achieving an overall accuracy of 81.56%. The model correctly identified 146 out of 179 cases. It exhibited a slight

bias toward predicting non-survival, with a higher recall for that class (0.87) compared to survivors (0.72). Analysis of the model's coefficients identified “Sex” (-1.25) as the most influential factor, followed by “Pclass” (-0.63) and “Age” (-0.43). Positive coefficients for “HasCabin” (0.35) and “Fare” (0.04) further confirmed that indicators of higher wealth and status positively impacted survival chances.

Limitations of the Study

The model achieved a solid 81.56% accuracy rate, but this figure should be read alongside some inherent constraints in the dataset. Its tendency to predict non-survival more reliably than survival reflects the underlying class imbalance, since non-survival was the more common outcome in the training data — a known challenge for classifiers trained on skewed class distributions (He & Garcia, 2009). In addition, while the selected features (e.g., Pclass, Sex, Age, Fare) offered strong predictive signal, the analysis was necessarily bounded by the data points available, which may have left out other situational factors that shaped individual survival outcomes during the disaster.

Recommendations

To further enhance predictive performance, future studies could explore:

1. **Feature Engineering:** Further refinement of variables such as "FamilySize" or "Title" might capture deeper nuances not fully addressed in the current model.
2. **Model Comparison:** Experimenting with more complex algorithms beyond Logistic Regression, such as Random Forests (Breiman, 2001) or Gradient Boosting, could potentially improve accuracy and better handle imbalanced data.
3. **Hyperparameter Tuning:** Implementing techniques like cross-validation and more rigorous hyperparameter optimization could refine the model's ability to generalize to new, unseen passenger data.

REFERENCE

Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley Publishing Company.

Encyclopaedia Britannica. (2024). *Titanic: Ocean liner*.
<https://www.britannica.com/topic/Titanic>

Kaggle. (2025). *About Kaggle*. <https://www.kaggle.com/about>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>

Bigelow, S. J., & Stedman, C. (2025, March 6). *What is data cleansing (data cleaning, data scrubbing)?* TechTarget.

Pilowsky, J. K., et al. (2024). Data cleaning for clinician researchers: Application and explanation of a data-quality framework. *Australian Critical Care*, 37(5), 827–833.
<https://doi.org/10.1016/j.aucc.2024.03.004>

Microsoft. (2025). *What is data visualization?* Microsoft Learn.

<https://learn.microsoft.com/power-bi/fundamentals/what-is-data-visualization>

Tableau. (2024). *What is data visualization?* <https://www.tableau.com/learn/articles/data-visualization>

APPENDIX

Codes

```
import importlib.util

import json

import os

import subprocess

import sys

import warnings

from pathlib import Path


warnings.filterwarnings("ignore")


def ensure_runtime_dependencies():

    required_packages = {

        "numpy": "numpy",

        "pandas": "pandas",

        "matplotlib": "matplotlib",

        "seaborn": "seaborn",

        "sklearn": "scikit-learn",

    }


    missing = [

        package
```

```

    for module_name, package in required_packages.items()

    if importlib.util.find_spec(module_name) is None

]

if not missing:

    return

print(f"Installing missing Python packages: {' '.join(missing)}")

try:

    subprocess.check_call([sys.executable, "-m", "pip", "install", *missing])

    importlib.invalidate_caches()

except subprocess.CalledProcessError as exc:

    raise RuntimeError(

        "Automatic dependency installation failed. "

        "Please run: "

        f"{sys.executable} -m pip install {' '.join(missing)}"

    ) from exc

ensure_runtime_dependencies()

import os

import subprocess

import sys

import matplotlib

```

```

try:

    matplotlib.use("TkAgg")
except Exception:

    try:

        matplotlib.use("Qt5Agg")
    except Exception:

        matplotlib.use("Agg")

import matplotlib.pyplot as plt

import numpy as np

import pandas as pd

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (

    accuracy_score,

    confusion_matrix,

    classification_report,

    ConfusionMatrixDisplay,

)


sns.set_style("whitegrid")

plt.rcParams["figure.dpi"] = 110

```

```

BASE_DIR = Path(__file__).resolve().parent

FIG_DIR = BASE_DIR / "figures"

OUT_DIR = BASE_DIR / "output"

UPLOAD_DIR = BASE_DIR # place train.csv, test.csv, gender_submission.csv here


FIG_DIR.mkdir(exist_ok=True)

OUT_DIR.mkdir(exist_ok=True)


stats_summary = {} # collects numbers we will quote in the written report


#
=====

=====

# TASK 1: DATA ACQUISITION

#
=====

=====

print("=" * 75)

print("TASK 1: DATA ACQUISITION")

print("=" * 75)


required_files = ["train.csv", "test.csv", "gender_submission.csv"]

missing = [

    f

    for f in required_files

```

```

    if not (UPLOAD_DIR / f).exists()
]
if missing:
    raise FileNotFoundError(
        f"Missing required file(s): {missing}. "
        f"Place train.csv, test.csv and gender_submission.csv in '{UPLOAD_DIR}' "
        f"(the Kaggle Titanic dataset) before running this script."
    )

train = pd.read_csv(UPLOAD_DIR / "train.csv")
test = pd.read_csv(UPLOAD_DIR / "test.csv")
gender_submission = pd.read_csv(UPLOAD_DIR / "gender_submission.csv")

print("\nTraining set dimensions:", train.shape)
print("Test set dimensions:", test.shape)
print("\nColumn names:", list(train.columns))
print("\nFirst five observations:\n", train.head())
print("\nData types:\n", train.dtypes)

stats_summary["train_shape"] = train.shape
stats_summary["test_shape"] = test.shape
stats_summary["columns"] = list(train.columns)

test_passenger_ids = test["PassengerId"].copy()

```

```

#
=====

=====

# TASK 2: DATA CLEANING

#
=====

=====

print("\n" + "=" * 75)

print("TASK 2: DATA CLEANING")

print("=" * 75)


# ---- 2.1 DETECT MISSING VALUES ----

print("\n--- 2.1 Detecting missing values ---")


missing_train = train.isnull().sum()

missing_train_pct = (missing_train / len(train) * 100).round(2)

missing_train_report = (

    pd.DataFrame({"missing_count": missing_train, "missing_pct": missing_train_pct})

    .query("missing_count > 0")

    .sort_values("missing_count", ascending=False)

)


missing_test = test.isnull().sum()

missing_test_pct = (missing_test / len(test) * 100).round(2)

missing_test_report = (

```



```

pd.DataFrame({"missing_count": missing_test, "missing_pct": missing_test_pct})

.query("missing_count > 0")

.sort_values("missing_count", ascending=False)

)

```

```

print("\nMissing values in TRAIN set:\n", missing_train_report)

```

```

print("\nMissing values in TEST set:\n", missing_test_report)

```

```

stats_summary["missing_train"] = train.isnull().sum().to_dict()

```

```

stats_summary["missing_test"] = test.isnull().sum().to_dict()

```

```

# ---- 2.2 DETECT DUPLICATE OBSERVATIONS ----

```

```

print("\n--- 2.2 Detecting duplicate observations ---")

```

```

duplicates_train = train.duplicated().sum()

```

```

duplicates_test = test.duplicated().sum()

```

```

print(f"\nDuplicate rows found in TRAIN set: {duplicates_train}")

```

```

print(f"Duplicate rows found in TEST set: {duplicates_test}")

```

```

if duplicates_train > 0:

```

```

    print("\nDuplicate rows in TRAIN (preview):\n", train[train.duplicated()])

```

```

if duplicates_test > 0:

```

```

    print("\nDuplicate rows in TEST (preview):\n", test[test.duplicated()])

```

```

stats_summary["duplicates_train"] = int(duplicates_train)

```

```
stats_summary["duplicates_test"] = int(dupes_test)
```

```
# ---- 2.3 HANDLE MISSING VALUES + REMOVE DUPLICATES ----
```

```
print("\n--- 2.3 Cleaning the data ---")
```

```
# Impute values are computed from the TRAINING set only. This avoids
```

```
# "data leakage": if we computed the median/mode from the combined
```

```
# train+test data, information from the test set would leak into how
```

```
# we treat the training set, which can make evaluation misleadingly
```

```
# optimistic.
```

```
age_median = train["Age"].median()
```

```
fare_median = train["Fare"].median()
```

```
embarked_mode = train["Embarked"].mode()[0]
```

```
def clean_data(df, age_median, fare_median, embarked_mode, label=""):
```

```
    """
```

```
    Cleaning decisions (explained):
```

```
    - Age (~20% missing in train, ~21% in test): imputed with the MEDIAN
```

```
      age computed from the TRAINING set only, to avoid leaking test
```

```
      information into the model. Median is used instead of mean because
```

```
      Age is right-skewed and less sensitive to outliers.
```

```
    - Embarked (2 missing in train): imputed with the MODE ('S'), since
```

```
      only two values are missing out of 891 and the port distribution is
```

```
      heavily dominated by Southampton.
```

```
    - Fare (1 missing in test): imputed with the median fare from training
```

data, for the same right-skew reasoning as Age.

- Cabin (~77% missing in train, ~78% in test): far too sparse to impute reliably, so instead of dropping the column outright we convert it into a binary indicator 'HasCabin' (1 = cabin recorded, 0 = missing). This preserves the signal that recording a cabin correlates with higher fare/class, while avoiding fabricated cabin values.
- Duplicates: detected and removed if present (none are expected in this dataset since each row is a unique passenger, but the check and removal step is kept for correctness and reproducibility).
- Dtypes: re-enforced after imputation, since filling NaNs can leave columns as generic 'object' dtype.

"""

```
df = df.copy()
```

```
n_before = len(df)
```

```
df["Age"] = df["Age"].fillna(age_median)
```

```
df["Embarked"] = df["Embarked"].fillna(embarked_mode)
```

```
df["Fare"] = df["Fare"].fillna(fare_median)
```

```
df["HasCabin"] = df["Cabin"].notnull().astype(int)
```

```
df.drop(columns=["Cabin"], inplace=True)
```

```
n_dupes = df.duplicated().sum()
```

```
df.drop_duplicates(inplace=True)
```

```
df["Pclass"] = df["Pclass"].astype(int)
```

```
df["Age"] = df["Age"].astype(float)
```

```
df["Fare"] = df["Fare"].astype(float)
```

```
n_after = len(df)
```

```
print(f"\n[{label}] Rows before cleaning: {n_before}")
```

```
print(f"[{label}] Duplicate rows removed: {n_dupes}")
```

```
print(f"[{label}] Rows after cleaning: {n_after}")
```

```
remaining_na = df.isnull().sum()
```

```
print(f"[{label}] Remaining missing values:\n{remaining_na[remaining_na > 0]}")
```

```
return df
```

```
train_clean = clean_data(
```

```
    train,
```

```
    age_median,
```

```
    fare_median,
```

```
    embarked_mode,
```

```
    label="TRAIN",
```

```
)
```

```
test_clean = clean_data(
```

```
    test,
```

```
    age_median,
```

```
    fare_median,
```

```

embarked_mode,

label="TEST",

)

print("\nImputation values used (derived from TRAIN only):")

print(f" Age median   = {age_median}")

print(f" Fare median   = {fare_median:.4f}")

print(f" Embarked mode = '{embarked_mode}'")

print(f"\nTrain shape after cleaning: {train_clean.shape}")

print(f"Test shape after cleaning: {test_clean.shape}")


stats_summary["age_median_used"] = float(age_median)

stats_summary["fare_median_used"] = float(fare_median)

stats_summary["embarked_mode_used"] = embarked_mode


#
=====

=====

# TASK 3: DATA VISUALISATION

#
=====

=====

print("\n" + "=" * 75)

print("TASK 3: DATA VISUALISATION")

print("=" * 75)

```

```

# Keep this section lightweight so the script reaches the text-based analysis
# and machine learning results promptly.

print("Generating a small set of summary figures...")

plt.figure(figsize=(8, 5))

plt.hist(train_clean["Age"], bins=30, color="#4F81BD", edgecolor="#2F4F4F")

plt.title("Distribution of Passenger Ages")

plt.xlabel("Age (years)")

plt.ylabel("Number of Passengers")

plt.tight_layout()

plt.savefig(FIG_DIR / "1_age_histogram.png")

try:

    if sys.platform.startswith("win"):

        os.startfile(FIG_DIR / "1_age_histogram.png")

    else:

        subprocess.run(["xdg-open", str(FIG_DIR / "1_age_histogram.png")], check=False)

except Exception:

    pass

plt.close()

plt.figure(figsize=(7, 5))

class_counts = train_clean["Pclass"].value_counts().sort_index()

bars = plt.bar(class_counts.index.astype(str), class_counts.values, color=["#4F81BD",
"#A6A6A6", "#2F5597"])

```

```

plt.title("Passenger Class Distribution")

plt.xlabel("Passenger Class")

plt.ylabel("Number of Passengers")

for bar in bars:

    plt.text(

        bar.get_x() + bar.get_width() / 2,

        bar.get_height() + 5,

        str(int(bar.get_height())),

        ha="center",

    )

plt.tight_layout()

plt.savefig(FIG_DIR / "2_pclass_bar.png")

try:

    if sys.platform.startswith("win"):

        os.startfile(FIG_DIR / "2_pclass_bar.png")

    else:

        subprocess.run(["xdg-open", str(FIG_DIR / "2_pclass_bar.png")], check=False)

except Exception:

    pass

plt.close()

numeric_cols = [

    "Survived",

    "Pclass",

    "Age",

```

```

    "SibSp",
    "Parch",
    "Fare",
    "HasCabin",
]

corr = train_clean[numeric_cols].corr()

plt.figure(figsize=(8, 6))

sns.heatmap(corr, annot=True, fmt=".2f", cmap="Blues", center=0, square=True)

plt.title("Correlation Heatmap")

plt.tight_layout()

plt.savefig(FIG_DIR / "3_correlation_heatmap.png")

try:
    if sys.platform.startswith("win"):
        os.startfile(FIG_DIR / "3_correlation_heatmap.png")
    else:
        subprocess.run(["xdg-open", str(FIG_DIR / "3_correlation_heatmap.png")], check=False)
except Exception:
    pass

plt.close()

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=train_clean,
    x="Age",

```



```

    y="Fare",
    hue="Survived",
    style="Pclass",
    palette=["#4F81BD", "#C0504D"],
    alpha=0.75,
    s=60,
)

plt.title("Age vs. Fare by Survival and Passenger Class")
plt.xlabel("Age (years)")
plt.ylabel("Fare (£)")
plt.tight_layout()
plt.savefig(FIG_DIR / "4_age_vs_fare_scatter.png")

try:
    if sys.platform.startswith("win"):
        os.startfile(FIG_DIR / "4_age_vs_fare_scatter.png")
    else:
        subprocess.run(["xdg-open", str(FIG_DIR / "4_age_vs_fare_scatter.png")], check=False)
except Exception:
    pass

plt.close()

pairplot = sns.pairplot(
    train_clean[["Age", "Fare", "Pclass", "Survived"]],
    hue="Survived",
    vars=["Age", "Fare", "Pclass"],

```

```

diag_kind="hist",

palette=["#4F81BD", "#C0504D"],

plot_kws={"alpha": 0.7},

)

pairplot.fig.suptitle("Pair Plot of Age, Fare, and Pclass by Survival", y=1.02)

pairplot.fig.tight_layout()

pairplot.fig.savefig(FIG_DIR / "5_pairplot_age_fare_pclass_survival.png")

try:

    if sys.platform.startswith("win"):

        os.startfile(FIG_DIR / "5_pairplot_age_fare_pclass_survival.png")

    else:

        subprocess.run(["xdg-open", str(FIG_DIR / "5_pairplot_age_fare_pclass_survival.png")],

check=False)

except Exception:

    pass

print(f"\nBasic EDA figures saved to '{FIG_DIR}/'")

#

=====

=====

# TASK 4: STATISTICAL ANALYSIS

#

=====

=====

```

```

print("\n" + "=" * 75)

print("TASK 4: STATISTICAL ANALYSIS")

print("=" * 75)


# ---- 4.1 Descriptive statistics ----

desc_numeric = train_clean[["Age", "Fare", "SibSp", "Parch"]].describe()

print("\nDescriptive statistics (numerical variables):\n", desc_numeric)

desc_numeric.to_csv(OUT_DIR / "descriptive_statistics.csv")


# ---- 4.2 Frequency distribution (categorical variables) ----

freq_sex = train_clean["Sex"].value_counts()

freq_survived = train_clean["Survived"].value_counts().rename(
    {0: "Did not survive", 1: "Survived"}
)

freq_pclass = train_clean["Pclass"].value_counts().sort_index()

freq_embarked = train_clean["Embarked"].value_counts()


print("\nFrequency distribution - Sex:\n", freq_sex)

print("\nFrequency distribution - Survived:\n", freq_survived)

print("\nFrequency distribution - Pclass:\n", freq_pclass)

print("\nFrequency distribution - Embarked:\n", freq_embarked)


survival_rate = train_clean["Survived"].mean()

survival_by_sex = train_clean.groupby("Sex")["Survived"].mean()

survival_by_class = train_clean.groupby("Pclass")["Survived"].mean()

```

```

print(f"\nOverall survival rate: {survival_rate:.4f}")

print("\nSurvival rate by sex:\n", survival_by_sex)

print("\nSurvival rate by class:\n", survival_by_class)


# ---- 4.3 Correlation analysis ----

print("\nFull correlation matrix:\n", corr)


corr_with_survival = corr["Survived"].drop("Survived").sort_values(ascending=False)

strongest_positive = corr_with_survival.idxmax()

strongest_positive_val = corr_with_survival.max()

strongest_negative = corr_with_survival.idxmin()

strongest_negative_val = corr_with_survival.min()


print(
    f"\nStrongest POSITIVE correlation with Survived: {strongest_positive} "
    f"(r = {strongest_positive_val:.3f})"
)

print(
    f"\nStrongest NEGATIVE correlation with Survived: {strongest_negative} "
    f"(r = {strongest_negative_val:.3f})"
)


# ---- 4.4 Three important statistical findings (printed for the report) ----

print("\nTHREE KEY STATISTICAL FINDINGS:")

```

```

print(
    "1. Sex was the single strongest predictor of survival: women survived "
    f"at {survival_by_sex['female']*100:.1f}% versus {survival_by_sex['male']*100:.1f}% "
    f"for men, a gap of {abs(survival_by_sex['female']-survival_by_sex['male'])*100:.1f} "
    "percentage points."
)

print(
    "2. Passenger class was strongly associated with survival: 1st class "
    f"passengers survived at {survival_by_class[1]*100:.1f}% compared with "
    f"{survival_by_class[3]*100:.1f}% for 3rd class."
)

print(
    "3. Fare (r = "
    f"{corr.loc['Fare','Survived']:.2f}) and HasCabin "
    f"(r = {corr.loc['HasCabin','Survived']:.2f}) were both positively correlated "
    "with survival, reinforcing that socio-economic status shaped survival odds, "
    "while Age (r = "
    f"{corr.loc['Age','Survived']:.2f}) was only weakly related."
)

stats_summary["survival_rate_overall"] = float(survival_rate)
stats_summary["survival_by_sex"] = survival_by_sex.to_dict()
stats_summary["survival_by_class"] = survival_by_class.to_dict()
stats_summary["strongest_positive_corr"] = {
    "feature": strongest_positive,

```

```

    "r": float(strongest_positive_val),
}

stats_summary["strongest_negative_corr"] = {

    "feature": strongest_negative,

    "r": float(strongest_negative_val),

}

stats_summary["corr_matrix"] = corr.round(3).to_dict()


#

=====

=====

# TASK 5: MACHINE LEARNING - FEATURE ENGINEERING

#

=====

=====

print("\n" + "=" * 75)

print("TASK 5: FEATURE ENGINEERING")

print("=" * 75)


def engineer_features(df):

    df = df.copy()

    df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

    df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

    df["Title"] = df["Name"].str.extract(r",\s*([^\.]*)\.")

    rare_titles = [

```

```

    "Lady",
    "Countess",
    "Capt",
    "Col",
    "Don",
    "Dr",
    "Major",
    "Rev",
    "Sir",
    "Jonkheer",
    "Dona",
]

df["Title"] = df["Title"].replace(rare_titles, "Rare")

df["Title"] = df["Title"].replace({"Mlle": "Miss", "Ms": "Miss", "Mme": "Mrs"})

return df


train_fe = engineer_features(train_clean)
test_fe = engineer_features(test_clean)


cat_cols = ["Sex", "Embarked", "Title"]

for col in cat_cols:

    le = LabelEncoder()

    combined = pd.concat([train_fe[col], test_fe[col]], axis=0).astype(str)

    le.fit(combined)

    train_fe[col] = le.transform(train_fe[col].astype(str))

```

```

test_fe[col] = le.transform(test_fe[col].astype(str))

# Select suitable predictor variables
feature_cols = [
    "Pclass",
    "Sex",
    "Age",
    "SibSp",
    "Parch",
    "Fare",
    "Embarked",
    "HasCabin",
    "FamilySize",
    "IsAlone",
    "Title",
]

print(f"\nPredictor variables selected: {feature_cols}")

print(
    "Rationale: Pclass, Sex, Fare and HasCabin proxy socio-economic status "
    "(shown in Task 4 to correlate with survival); Age, SibSp, Parch, "
    "FamilySize and IsAlone capture demographic/family context; Embarked "
    "and Title add small additional signal. PassengerId, Name and Ticket "
    "are excluded as identifiers with no predictive meaning."
)

```



```

X = train_fe[feature_cols]

y = train_fe["Survived"]

X_test_final = test_fe[feature_cols]


# Split the dataset into training and testing sets

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTraining set size: {X_train.shape[0]} passengers")

print(f"Validation (hold-out test) set size: {X_val.shape[0]} passengers")


scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_val_scaled = scaler.transform(X_val)

X_test_final_scaled = scaler.transform(X_test_final)


#
=====

=====

# TASK 6: MACHINE LEARNING - LOGISTIC REGRESSION (Model Evaluation)

#
=====

=====

print("\n" + "=" * 75)

```

```
print("TASK 6: MACHINE LEARNING (LOGISTIC REGRESSION MODEL  
EVALUATION)")
```

```
print("=" * 75)
```

```
# 3. Train a Logistic Regression classifier
```

```
log_reg = LogisticRegression(max_iter=1000, random_state=42)
```

```
log_reg.fit(X_train_scaled, y_train)
```

```
# 4. Predict the testing data
```

```
y_pred = log_reg.predict(X_val_scaled)
```

```
# 5. Compute accuracy, confusion matrix, classification report
```

```
acc = accuracy_score(y_val, y_pred)
```

```
cm = confusion_matrix(y_val, y_pred)
```

```
report = classification_report(  
    y_val, y_pred, target_names=["Did not survive", "Survived"]  
)
```

```
print(f"\nModel Accuracy: {acc:.4f}")
```

```
print("\nConfusion Matrix:\n", cm)
```

```
print("\nClassification Report:\n", report)
```

```
stats_summary["logreg_accuracy"] = float(acc)
```

```
stats_summary["logreg_confusion_matrix"] = cm.tolist()
```

```

plt.figure(figsize=(6, 5))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Did not survive", "Survived"],
)

disp.plot(cmap="Blues", values_format="d")

plt.title("Confusion Matrix - Logistic Regression")

plt.tight_layout()

plt.savefig(FIG_DIR / "7_confusion_matrix.png")

try:
    if sys.platform.startswith("win"):
        os.startfile(FIG_DIR / "7_confusion_matrix.png")
    else:
        subprocess.run(["xdg-open", str(FIG_DIR / "7_confusion_matrix.png")], check=False)
except Exception:
    pass

plt.close()

# Coefficients (for discussion of which features drive predictions)
coef_series = pd.Series(log_reg.coef_[0], index=feature_cols).sort_values()

print("\nLogistic Regression coefficients (standardized features):\n", coef_series)

plt.figure(figsize=(8, 6))

coef_series.plot(kind="barh", color=["#4F81BD", "#A6A6A6", "#5B9BD5", "#7F7F7F",
"#2F5597", "#6C8EBF", "#C9D9F2", "#B4B4B4", "#4A7FB8", "#4F6180", "#1F497D"])

```

```

plt.title("Logistic Regression Coefficients (Standardized Features)")

plt.xlabel("Coefficient value")

plt.ylabel("Feature")

plt.axvline(0, color="black", linewidth=0.8)

plt.tight_layout()

plt.savefig(FIG_DIR / "8_logreg_coefficients.png")

try:

    if sys.platform.startswith("win"):

        os.startfile(FIG_DIR / "8_logreg_coefficients.png")

    else:

        subprocess.run(["xdg-open", str(FIG_DIR / "8_logreg_coefficients.png")], check=False)

except Exception:

    pass

plt.close()


# 6. Predict on the official Kaggle test set + build submission file

test_predictions = log_reg.predict(X_test_final_scaled)

submission = pd.DataFrame({"PassengerId": test_passenger_ids, "Survived":
test_predictions})

submission.to_csv(OUT_DIR / "submission.csv", index=False)


baseline_acc = accuracy_score(gender_submission["Survived"], test_predictions)

print(

    f"\nSubmission file saved. Agreement with gender_submission.csv baseline: "

    f"{baseline_acc:.4f}"

```

```
)

stats_summary["kaggle_test_agreement_with_baseline"] = float(baseline_acc)


# Save the stats summary for use when writing the report
with open(OUT_DIR / "stats_summary.json", "w") as f:

    json.dump(stats_summary, f, indent=2, default=str)


print("\n" + "=" * 75)

print("ALL TASKS COMPLETE")

print("=" * 75)
```